Supplementary Information

FanarVSE: Fast, Memory-Light GPU Vector Search with Error Bars from Coding Theory

Mohammed Mohsen

Numbers without the prefix S (sections, theorems, lemmas, tables, figures) refer to the main text.

Supplementary Note 1. Structured rotations

Structured rotations. The theorem needs a Haar rotation; fast systems use structured ones such as random signs plus Walsh–Hadamard transforms [1]. We measured the standardised error of FanarVSE's 4-bit stage-2 code on 200,000 query–candidate pairs (1,000 queries, 200 candidates each, 998k vectors) under six rotations (Table S1). With a dense Haar rotation the measured law matches Corollary 1 (standard deviation 1.0015, tail masses 4.59% and 0.273%). One round of random signs plus a block Walsh–Hadamard transform on blocks of 512 coordinates, three rounds with random permutations, and block-diagonal Haar rotations with blocks of 64 or even 8 coordinates are indistinguishable from it, and so are recall and exact reads. Only removing the rotation changes the law: the error acquires a bias ($\mathbb{E}Z = -0.065$) and heavier tails (5.06%, 0.324%), though on these embeddings recall is unchanged because their residual coordinates are already close to isotropic. The guarantees below therefore hold as stated for Haar rotations and, empirically, for the structured rotations in Table S1.

Table S1: Standardised stage-2 error Z under different rotations (4-bit code, 200k query–candidate pairs at 998k vectors) and the resulting search quality ($n_p = 32$, $K = 400$, $z = 2$). The first row is the exact law of Corollary 1.

Rotation	std(Z)	$P(\lvert Z\rvert>2)$	$P(\lvert Z\rvert>3)$	excess kurt.	Recall@10	Reads/query
Exact law, $d = 1536$	1	4.55%	0.267%	−0.004	–	–
Dense Haar	1.002	4.59%	0.273%	−0.002	0.9922	5.85
Signs + block WHT (1 round)	1.002	4.56%	0.260%	−0.010	0.9922	5.80
(Permutation, signs, block WHT) ×3	0.998	4.46%	0.252%	−0.019	0.9922	5.78
Block-diagonal Haar, blocks of 64	1.000	4.52%	0.282%	0.000	0.9921	5.84
Block-diagonal Haar, blocks of 8	1.004	4.64%	0.270%	−0.012	0.9922	5.84
None	1.021	5.06%	0.324%	−0.010	0.9922	5.83

Supplementary Note 2. When the codebook order can fail

When the order can fail. The corollary ranks codebooks by c only when the rotated coordinates look Gaussian, which is what the rotation provides. We repeated the IVF comparison with and without the rotation on the four embedding sets and on two synthetic sets built to break it: heavy-tailed vectors (Student-t with 3 degrees of freedom, 128 dimensions) and sparse vectors (16 non-zero Gaussian entries out of 256). On all four embedding sets the residual coordinates are close to Gaussian even without the rotation (median excess kurtosis between −0.03 and 0), and the predicted order holds in all eight settings (one tie: 0.7633 vs. 0.7631 on arXiv). Without the rotation, the synthetic sets have kurtotic coordinates (median excess kurtosis 3.1 and 34). On the heavy-tailed set the order survives but every codebook loses (recall@10 0.19–0.35 without the rotation, 0.44–0.50 with it). On the sparse set the order breaks: BW16 has the larger c (0.775 vs. 0.740 for E8) but the lower recall@10 (0.321 vs. 0.341), and the hypercube collapses ($c = 0.29$). The rotation restores both the Gaussian shape (excess kurtosis −0.03) and the predicted order (recall 0.590, 0.615, 0.630). The diagnostic we recommend is therefore the excess kurtosis of the coded coordinates; when it is far from zero, c should not be used to compare codebooks, and the rotation should be applied.

Supplementary Note 3. Error covariance of decoders

To test the mechanism rather than only its outcome, we measured the covariance $E = \mathbb{E}[\varepsilon\varepsilon^\top]$ of the decoded error $\varepsilon = \hat{o} - o$ ($\hat{o}$ is the vector the estimator multiplies q by) on 20k OpenAI vectors, the query second moment Σ_q from 2,000 queries, and the error on the pairs that decide rankings: each of 400 queries with its true top-50 (Table S2). For every unbiased decoder of Theorem 1 (sign, 4-bit Lloyd–Max, trellis), the ranking error $\mathrm{tr}(\Sigma_q E)$ equals the isotropic prediction $\mathrm{tr}(\Sigma_q)\mathrm{tr}(E)/d$ to within 1% ($\rho = 0.99$), the share of $\mathrm{tr}(E)$ in the 50 leading principal directions of the data equals their share of the dimensions, and lowering $\mathrm{tr}(E)$ (sign $\to$ trellis) lowers the error variance across a query's top-50 and raises recall. The Bussgang decoders lower $\mathrm{tr}(E)$ but concentrate 9–16% of it in the 50 leading directions (3.3% expected) and are 2.4–4.0$\times$ worse than isotropic on Σ_q; on the ranking-relevant pairs their error variance is 1.4–1.5$\times$ that of the plain sign decoder, and recall falls accordingly. In 256 PCA dimensions the raw Bussgang decoder even lowers the pair error variance but still loses recall (0.585 vs. 0.645), because its error is biased and correlated with the score, which Theorem 1 rules out for unbiased decoders. A controlled experiment with synthetic errors (Section 6.4) isolates the mechanism: at equal MSE, recall varies from 0.49 to 0.97 with the alignment of the error, and it is determined by $\mathrm{tr}(\Sigma_q E)$ ($R^2 = 0.998$), not by the MSE ($R^2 = 0.20$).

Table S2: Error covariance of decoders on OpenAI-1536 (228k vectors). $\rho = \mathrm{tr}(\Sigma_q E)/(\mathrm{tr}(\Sigma_q)\mathrm{tr}(E)/D)$ is 1 for isotropic error; "top-50 share" is the fraction of $\mathrm{tr}(E)$ in the 50 leading principal directions (isotropic: $50/D$); "pair var." is the error variance across each query's true top-50, and the last column the fraction of the true top-10 ranked in the top 10 among those 50.

Code (dimensions D)	Decoder	MSE $\mathrm{tr}(E)$	ρ	top-50 share	pair var. (10^{-4})	Recall
Sign (1536)	unbiased (Thm. 1)	0.570	0.99	3.2%	2.67	0.834
Sign (1536)	Bussgang LMMSE	0.188	2.39	9.1%	3.90	0.788
Sign (1536)	Bussgang, unbiased scale	0.232	3.98	16.0%	3.82	0.794
4-bit Lloyd–Max (1536)	unbiased	0.010	0.99	3.2%	0.048	0.975
Sign (256 PCA)	unbiased	0.571	0.99	19.4% (19.5)	11.4	0.645
Trellis L=10 (256 PCA)	unbiased	0.383	1.00	19.5% (19.5)	8.08	0.665
Sign (256 PCA)	Bussgang LMMSE	0.305	1.26	27.9% (19.5)	9.36	0.585

Supplementary Note 4. Identification estimate and the effective dimension

Table S3 gives the identification estimate of Section 3.3 under different definitions of the effective dimension.

Table S3: Identification-rate estimate $B_{\mathrm{ID}} = n_{\mathrm{eff}} \log_2(1/\tau)$ for OpenAI-1536 (228k vectors, $k = 10$, median $\tau = 0.459$; in brackets the range over the 10th–90th percentile of τ) under different definitions of the effective dimension.

Definition of n_{eff}	n_{eff}	Bits per vector
Participation ratio $(\sum\lambda)^2/\sum\lambda^2$	124	140 (104–169)
Exponential of spectral entropy	342	384 (287–464)
Components for 90% of the variance	433	486 (363–587)
Components for 95% of the variance	636	714 (533–863)
Components for 99% of the variance	1,056	1,186 (884–1,432)

Supplementary Note 5. Engineering details

Small batches. The batched path builds its work list on the GPU but needs the host to size it, and every such synchronisation costs a few hundred microseconds. For up to eight queries FanarVSE therefore uses a second stage-1 kernel with one program per (query, probed cell), which needs no work list, and for up to 32 queries a

kernel with one program per (cell, block) that reads a dense query-by-cell probe mask. These kernels decode trellis codes with a variant that loads each row's nine code words once and builds the windows in registers; it decodes the whole database in 1.39 ms instead of 2.56 ms, which shortens small batches by 25–30%. It slowed the work-list kernels of the batched path, where registers are scarce, but in the fixed-grid kernels described next it shortens stage 1 by 11–16%, so only the work-list path, now a fallback, keeps the original decoder. Both use the same guessed threshold and count check. Stages 1–3 then contain no host synchronisation, so the whole search is captured once as a CUDA graph and replayed; the host reads back only the exactness flags. A query that fails the count check is recomputed on the batched path.

A synchronisation-free batch path. The work list of the batched path has a size that depends on the queries, so building it needs several host synchronisations (sizing the list, finding the largest buffer); at a batch of 500 the GPU was idle for a large part of stage 1. We replaced it by a fixed work list with one program for every (cell, block) of the index. For each call, the probes are sorted by cell with fixed-size operations (a stable sort, per-cell counts by a scatter-add, an exclusive prefix sum), and a program whose cell no query probes exits at once. The emit buffer has a fixed width $M = \max(4{,}096, 16K)$, and a query that emitted fewer than K or more than M items is flagged and recomputed alone on the work-list path, which keeps every result exact (Lemma 3); this happened for 0–4 of 2,000 queries. The whole search is then one CUDA graph at every batch size, with a single synchronisation for the flags. It raised throughput by 5–7% at the same recall; most of stage 1's time is GPU work after all.

The NVMe tier. We measured unbuffered random reads on the laptop's NVMe drive: about 82k, 88k and 84k IOPS for 512 B, 4 KB and 8 KB reads with synchronous requests from 16–32 threads. Cost is per request, not per byte, so reading lower-precision vectors from SSD saves nothing; what matters is the number of reads, which the error bars minimise. FanarVSE-v1 issued one blocking read per vector from Python threads and reached 3.0k to 4.4k QPS. The current reader is a small C library (compiled with the TCC compiler that ships with Triton) that reads a whole batch with overlapped I/O and an I/O completion port, keeping up to 128 requests in flight: it reaches 127k random 4 KB reads per second (54k for the threaded Python reader on the same requests), and it runs in a host thread so that the reads of one batch overlap the GPU stages of the next.

A fully GPU-resident variant. When no host RAM or NVMe tier is available, the exact stage can be replaced by a finer refinement code on the GPU, following successive refinement [2]. In an earlier configuration with a 768-dimension sign-code stage 1 and an 8-bit refinement (1,644 B/vector in total, no host data), FanarVSE reached recall 0.9915 at 5,328 QPS on 998k vectors ($n_p = 32$, $K = 100$); with a 4-bit refinement alone recall plateaus at 0.956, the resolution limit discussed above.

Supplementary Note 6. Tightening σ

Tightening σ. The conservative $\|q\|$ costs reads, and $\|q_\perp\|^2 = \|q\|^2 - \langle q, o\rangle^2/\|o\|^2$ can be estimated by plugging in the stage-2 estimate of $\langle q, o\rangle$ and a stored residual norm $\|o\|$ (2 B/vector). With this plug-in σ, the standardised error of the intermediate engine of Table 4 has standard deviation 1.008, as Theorem 1 predicts, instead of 0.914; reads fall from 6.61 to 5.56 per query at $z = 2$ and from 17.9 to 14.7 at $z = 4.9$, recall is unchanged (0.9929), and the share of queries with an uncovered candidate at $z = 4.9$ is unchanged (0.05%). The plug-in makes σ depend on the estimate, so the certificate above does not apply to it. A certified variant replaces the estimate by a lower confidence bound, per tier:

Corollary S1 (Certified plug-in)**.** *Store $\|o_t\|$ per tier, let $\hat{s}_t$ be tier t's estimate of $s_t = \langle q_t, o_t\rangle$ and $\sigma_t^q = \|q_t\|\sigma_t$ its engine standard deviation, where $\sigma_t = \|o_t\|\sqrt{1-c_t^2}/(c_t\sqrt{d_t - 1})$ is the stored per-vector factor ($\|q_t\|$ in place of $\|q_{t\perp}\|$). Set $u_t^2 = \|q_t\|^2 - \max(0, |\hat{s}_t| - z_1\sigma_t^q)^2/\|o_t\|^2$ and $\sigma'^2 = \sum_t u_t^2\sigma_t^2$. Then $P(|\hat{s} - s| > z_2\sigma') \le 2Te^{-z_1^2/2} + 2e^{-z_2^2/2}$ for T tiers.*

Proof. With probability at least $1 - 2Te^{-z_1^2/2}$ every tier satisfies $|\hat{s}_t - s_t| \le z_1\sigma_t^q$ (Corollary 1 applied to tier t with the wider σ_t^q); on this event $|s_t| \ge |\hat{s}_t| - z_1\sigma_t^q$, so $\|q_{t\perp}\|^2 = \|q_t\|^2 - s_t^2/\|o_t\|^2 \le u_t^2$ and σ' bounds the true standard deviation. The sub-Gaussian bound of Corollary 3 with σ' then holds on the event, and the union bound adds the two failure probabilities. □

With $z_1 = 5.6$ and $z_2 = 4.95$ the per-query bound at $K = 400$ is 0.46%, slightly below the 0.49% of the default certificate, and the reads fall from 17.9 to 15.5 per query (p99 41 to 36) at the same recall; 0.05% of queries had an uncovered candidate in both cases. The stored norms cost 12 B/vector (six fp16 values). We keep $\|q\|$ as the default because the uncertified setting $z = 2$ gains more from the simple plug-in, and the certified gain is 13%.

Supplementary Note 7. Choosing the parameters

Choosing the parameters. In our experience the parameters can be set in this order. (1) R: 256 principal dimensions suffice up to recall 0.999 on the OpenAI data, and 384 above it; flatter spectra need more (512 for DINOv2, Section 6.9). (2) The refinement size: 3,840 bits (570 B in total) gives about six exact reads per query, and 2,560–3,072 bits (406–470 B) give 11–15 reads at the same throughput when reads are served from host RAM (Table S5); prefer the larger code when reads come from NVMe. (3) Cells: about $\sqrt{N}$ to $4\sqrt{N}$ (4,096 at 1M). (4) K and n_p: recall is set by the candidate-set ceiling, which the error bars reach in every configuration, so tune K and n_p on a few hundred queries against exact search; $K \approx 40k$–$80k$ and $n_p \approx N_{\text{cells}}/8$ to $N_{\text{cells}}/4$ give recall 0.99–0.997 at $k = 10$, and K must grow with k (K = 1,500–3,000 for $k = 100$). (5) z: 2 by default, 4.9 (tiered code, $K = 400$) when a per-query certificate is required; the reads grow by about 2.7×, the throughput falls by about 7%.

Supplementary Note 8. Same operating system for both engines

Same operating system for both? To check that the operating-system split does not favour FanarVSE, we also ran FanarVSE itself in WSL2 with the same PyTorch and Triton versions (Table S4). Its throughput there is within 2% of native Windows (slightly higher at $n_p = 32$), host-to-device and device-to-host bandwidth and cuBLAS GEMM throughput agree to 1.5%, and asynchronous kernel launches are faster in WSL2 (3.2 vs. 6.5 µs). Only random gathers from host memory are slower in WSL2 (5.7 vs. 2.3 ms for 12k rows), which affects FanarVSE's stage 3, not the baselines.

Table S4: FanarVSE and micro-benchmarks on native Windows and in WSL2 (same GPU, driver, PyTorch 2.6 and Triton 3.2; 998k vectors).

Measurement	Windows	WSL2
FanarVSE n_p=8, K=200 (recall 0.962), QPS	10,642	10,571
FanarVSE n_p=32, K=200 (recall 0.985), QPS	7,117	7,240
FanarVSE n_p=32, K=400 (recall 0.992), QPS	5,295	5,383
FanarVSE latency, batch 1 / 16 (ms)	3.5 / 6.9	3.6 / 6.9
Kernel launch, asynchronous / with synchronisation (µs)	6.5 / 38	3.2 / 39
Pinned copy host→device / device→host (GB/s)	10.2 / 9.3	10.2 / 9.4
fp16 GEMM $2000 \times 1536 \times 10^5$ (TFLOP/s)	17.7	17.9
Gather of 12k random fp16 rows from host RAM (ms)	2.3	5.7

Supplementary Note 9. Smaller refinement codes

Smaller refinement codes. The refinement size trades GPU memory for exact reads (Table S5). Recall does not depend on it, since the error bars reach the candidate-set ceiling at every size; only the number of ambiguous candidates changes, as σ grows with fewer bits. Since zero-copy reads are cheap, codes of 338–470 B/vector are as fast as 570 B (between 4% slower and 7% faster) with 11–29 reads per query. With 262 B/vector (1 bit per coordinate in the refinement) the engine still runs at 16,645 QPS at recall 0.992, 2.1–2.2× the throughput of IVF-RaBitQ at 0.992 with 7.1× less GPU memory, but it reads 84 full vectors per query, which suits host RAM better than NVMe. Above 570 B the larger refinement costs more to read than the reads it saves. We keep 570 B as the default because its six reads per query also suit the NVMe tier.

Table S5: GPU memory per vector (including the shared tables) against throughput: refinement codes from 1,536 to 4,608 bits (final engine, stage 1 fixed at $R = 256$; 998k vectors, $k = 10$, batch 500, $z = 2$; mean of three runs).

GPU B/vec	Tiers	n_p=384, K=400 (recall 0.992)		n_p=1,024, K=800 (recall 0.998)	
		QPS	Reads/query	QPS	Reads/query
262	1	16,645	84.0	12,272	91.9
338	4	21,832	28.6	14,509	29.0
406	5	23,189	15.0	14,524	15.1
470	5	23,202	11.0	14,508	11.0
570	6	22,747	6.6	13,565	6.7
666	6	19,895	4.9	11,696	4.9

Supplementary Note 10. Seeds

Seeds. We rebuilt the index with five random seeds (stage-1 rotation, stage-2 rotation and coarse quantizer). Recall@10 ranged from 0.9825 to 0.9854 at $K = 200$ and from 0.9920 to 0.9931 at $K = 400$, throughput varied by 1.3%, and the read counts by 0.13 per query (5.73–5.86; p99 15–16; maximum at most 22). The standard deviation of the standardised error, with the engine's conservative $\|q\|$ in place of $\|q_\perp\|$, was 0.890–0.895 for every seed. These numbers are for FanarVSE-v1. For the final engine, with five seeds for every random choice (rotations, tiered rotations, both coarse quantizers), recall@10 ranged from 0.9906 to 0.9919 at $n_p = 384$, $K = 400$, from 0.9950 to 0.9959 at $n_p = 768$, $K = 400$ and from 0.9979 to 0.9986 at $n_p = 1{,}024$, $K = 800$; throughput varied by 0.5%, 0.9% and 0.8%, and reads by at most 0.10 per query (6.59–6.69; p99 16–18; maximum at most 28).

Supplementary Note 11. Direct tests of the theory

These tests are summarised in Section 6.4 and Figure 6.

The exact law, down to eight dimensions. At $d = 1536$ the law of Corollary 1 and a Gaussian agree to three decimals, so the measurements of Table S1 cannot tell them apart. We therefore projected 20,000 OpenAI vectors and 500 queries onto their top D principal components ($D = 8$ to 1,536), coded the rotated vectors with five codebooks (hypercube, E8, 2- and 4-bit Lloyd–Max and the 1-bit trellis) and measured Z on all 10 million query–vector pairs, with three Haar rotations per setting. For the four codebooks other than the hypercube, tail masses, kurtosis and spread follow the exact law at every D (Table S6, Figure 6a,b). At $D = 8$, 3.03% of the errors exceed 2σ (law 3.00%, Gaussian 4.55%), none exceeds 3σ because the law's support ends at $\sqrt{7} \approx 2.65$ (a Gaussian puts 0.27% there), and the excess kurtosis is -0.671 (law -0.667). The Kolmogorov–Smirnov distance to the exact law is 6× smaller than to the Gaussian at $D = 8$ and 3× smaller at $D = 16$. The law also holds on each query's true top-100, the pairs that decide rankings (standard deviation 0.98–1.02 at every D). There are two departures, both at $D \leq 16$. With 200,000 sampled pairs the KS test still detects a small deviation from the law in 5 of 12 runs at $D = 8$ and 3 of 12 at $D = 16$, and the hypercube code shows a bias of up to 0.05σ. Both come from coding all vectors with one rotation, whereas the theorem averages over the rotation; neither appears in the per-pair test below.

One pair, many rotations. Theorem 1 is a statement about one fixed pair (q, o) and a random rotation, and we tested it in that form. For each $D \in \{16, 64, 256, 1536\}$ we took 60 pairs (20 queries, each with its true nearest neighbour, its 10th neighbour and a random vector) and coded each pair under 1,000 independent Haar rotations. We ran 11 settings (sign, 4-bit Lloyd–Max and trellis codes). In 10 of them the share of pairs whose mean Z differs from zero at $p < 0.01$ was 0–3.3% (1% expected) and Fisher's combined p-value for bias was 0.16–0.97; in all 11 the per-pair standard deviations lay between 0.93 and 1.06 (sampling error 0.022). The exception was the 4-bit code at $D = 1536$ in single precision (combined $p = 0.0007$); repeated in double precision with 1,000 fresh rotations, it shows no bias (no pair at $p < 0.01$, combined $p = 0.41$), so we attribute it to rounding in the single-precision rotations, although a new draw of rotations alone could also explain it. At $D = 16$ the test separates the two laws: for the 4-bit code a Gaussian is rejected (combined $p = 10^{-5}$) and the exact law is not ($p = 0.05$).

Table S6: The exact error law (Corollary 1) against a Gaussian, on OpenAI vectors projected to D principal dimensions. Each row averages 12 runs (E8, 2- and 4-bit Lloyd–Max and trellis codes, three Haar rotations each; at $D = 1536$, 9 runs without the trellis code), 10 million query–vector pairs per run. Measured / exact law; the Gaussian values are 4.55%, 0.270% and 0.

D	$P(\|Z\| > 2)$ (%)		$P(\|Z\| > 3)$ (%)		Excess kurtosis	KS distance
	measured	law	measured	law	measured / law	to law / to Gaussian
8	3.03	3.00	0.000	0.000	-0.671 / -0.667	0.0040 / 0.0245
16	4.09	4.06	0.043	0.043	-0.356 / -0.353	0.0036 / 0.0122
32	4.36	4.35	0.147	0.146	-0.183 / -0.182	0.0023 / 0.0061
64	4.46	4.46	0.208	0.208	-0.092 / -0.092	0.0017 / 0.0031
256	4.52	4.53	0.254	0.254	-0.023 / -0.023	0.0022 / 0.0022
1536	4.55	4.55	0.269	0.267	-0.003 / -0.004	0.0019 / 0.0019

Conditional on c, rank and $\|q_\perp\|$. On 200,000 stage-2 pairs at 998k vectors, Z is uncorrelated with the true score (-0.005) and with c (-0.0004). Its standard deviation is 0.997–1.008 within each quintile of c and each quartile of $\|q_\perp\|$, and 0.985–1.007 for the true nearest neighbour, the 2nd–10th, the 11th and the remaining candidates, so the error does not depend on whether a candidate is a true neighbour. The engine's interval, which uses $\|q\|$ for $\|q_\perp\|$, is wider than necessary by a factor of $1/0.73$ for the nearest neighbour and $1/0.90$ for distant candidates, since near neighbours have the smallest $\|q_\perp\|$. The errors of one query's candidates are weakly correlated: the variance of a query's mean Z is $1.69/K$ instead of $1/K$, a mean pairwise correlation of 0.003.

The two lemmas. Lemma 4 held without exception: over 2,000 queries, $K \in \{200, 400\}$ and eight values of z, all 19,803 query runs whose intervals covered every candidate returned exactly the top-10 of the candidate set. (Coverage is sufficient, not necessary: 12,191 of the 12,197 runs with an uncovered candidate were exact as well, because a miss matters only near the k-th score.) For the block-max selection lemma we computed every stage-1 estimate in the probed cells for 400 queries. The selected candidates equal the exact top-K by estimate for 100% of items at $K = 200$ and 99.9988% at $K = 400$; every differing item lies within $2 \cdot 10^{-3}$ of the K-th estimate, a tie broken differently by the fp16 block maxima.

The union bound. Figure 6d compares the share of queries with at least one uncovered candidate with $K\,P(|Z| > z)$. With the exact σ the observed shares track the bound but lie slightly above it at large z (at $K = 400$: 2.9% against 2.4% at $z = 4$, 0.60% against 0.49% at $z = 4.36$). Each difference is within the sampling error of 2,000 queries (Poisson $p = 0.1$–0.3), but they all point the same way. The bound holds in expectation over the rotation and one index is one draw, so we do not claim it holds with the exact σ for every index. With the engine's interval it holds with a margin of 3–6×: at the certified setting ($z = 4.36$, $K = 400$) 0.10% of queries had an uncovered candidate against the bound of 0.5%, and none did at $z = 5$.

Error isotropy, controlled. The evidence for the error-isotropy principle in Section 3.2 comes from real decoders, which differ in more than their error covariance. To test the principle directly, we added synthetic unbiased Gaussian errors to the exact 228k OpenAI vectors, with covariance $E \propto V\mathrm{diag}(\lambda^\gamma)V^\top$ in the data's principal basis, and ranked by $\langle q, o + e\rangle$. The exponent γ ran from -1 (error in low-variance directions) through 0 (isotropic) to 2 (error concentrated in the leading directions), and the MSE $\mathrm{tr}(E)$ from 0.05 to 0.8. Across the 30 settings, recall@10 is a function of $\mathrm{tr}(\Sigma_q E)$ alone: a cubic fit in $\log \mathrm{tr}(\Sigma_q E)$ explains $R^2 = 0.998$ of its variance (Spearman -1.00), while one in the MSE explains 0.20 (Figure 6c). At an MSE of 0.2, recall is 0.968 for $\gamma = -1$ and 0.491 for $\gamma = 2$. Conversely, $\gamma = -1$ at MSE 0.8 and the isotropic error at MSE 0.05, 16× less, have similar ranking errors (2.0 and 3.3×10^{-5}) and similar recall (0.939 and 0.926). Ranking quality is therefore governed by the error along query directions, not by the MSE. The converse of the Bussgang result also follows: an unbiased code that placed its error in low-variance directions would rank better at equal MSE. Such shaping costs bits, and we leave codes that do it to future work.

The identification estimate, out of sample. The estimate of Section 3.3 was compared with codes on the same data it was computed from. We tested it on GloVe-100 and across k, with the two datasets at the same size (228,000 vectors, the GloVe set subsampled), the same queries per dataset (2,000) and a fixed number K

of candidates (Table S7). For each case we measured the bits a stage-1 code needs to keep 95% of the true top-k among its top K estimates, using the best of sign, trellis and 2-bit Lloyd–Max codes on R rotated principal dimensions at each size. The estimate falls between the bits needed with $K = 2{,}000$ and with $K = 200$ candidates in all six cases. It predicts correctly that more bits are needed for larger k at a fixed K, and it is within a factor of two of the bits needed whenever $K \geq 20k$. It has two failures. First, it does not model the candidate budget, which changes the bits needed by a factor of 1.6 to 5. Second, it does not reliably rank the two datasets: it predicts that GloVe needs fewer bits than OpenAI (100 against 140 for $k = 10$), which holds at $K = 200$ (158 against 181) but not at $K = 2{,}000$ (97 against 76). We therefore keep it as an order-of-magnitude guide when $K \geq 20k$, and we do not use it to choose between datasets or candidate budgets.

Table S7: Identification estimate B_{ID} (participation-ratio n_{eff}, median τ_k) against the measured bits per vector a stage-1 code needs to keep 95% of the true top-k among its top K estimates (228k vectors per dataset, 2,000 queries; interpolated between code sizes).

Dataset (n_{eff})	k	B_{ID}	Measured, $K = 2{,}000$	Measured, $K = 200$
OpenAI-1536 (124)	1	101	61	115
	10	140	76	181
	100	180	125	620
GloVe-100 (89)	1	76	65	103
	10	100	97	158
	100	126	107	$> 200^{*}$

*The largest code tested (2 bits on all 100 dimensions) keeps 83.6%.

Supplementary Note 12. Graph front-end

A graph front-end for throughput. At recall 0.99 stage 1 scores about 94k codes per query. A proximity graph can reach the same candidates with far fewer scores, so we built an optional graph mode that replaces only stage 1 and keeps stages 2 and 3 unchanged. The graph is the exact 64-nearest-neighbour graph (232 s by brute force on the GPU), pruned with the relative-neighbourhood rule (Vamana with $\alpha = 1$ [3]) to 32 out-edges plus reverse edges. Navigation uses a separate 256-byte int8 code of $x - c$ projected on the 256 leading principal directions, with one fp16 scale per vector. One 128-thread block per query runs a beam search written in CUDA: it expands the two best unexpanded beam entries per step, scores their neighbours with integer dot products (dp4a), keeps the beam in shared memory, merges new candidates by bitonic sort and merge path, and filters revisits with a 4,096-slot direct-mapped table of visited ids plus a check against the ids in the beam. The search starts from the 64 best of 8,192 sampled vectors and hands its best $K = 128$ candidates to stage 2.

In simulation the walk needs about 7k scores per query for a candidate-set ceiling of 0.99, 13× fewer than the cell scan. End to end the gain is much smaller (Table 10): 28,770 QPS at recall 0.9905 and 23,336 QPS at 0.9926, against 22,649 QPS at 0.992 for the default engine, so about 1.2–1.3× near recall 0.99 and parity above it. Each graph step is a dependent random read, while the cell scan reads code blocks in order and shares them across the batch. Cycle counters in the kernel attribute 82–94% of its time to scoring neighbours, and the walk saturates at about 2.5×10^8 neighbour scores per second on this GPU. Neither smaller navigation codes (1-bit, 64 B, with popcount scoring), other thread layouts, one warp per query, nor larger batches (250–2,000) raised that rate. The graph mode also costs memory: the navigation codes and the graph add 386 B per vector to the engine's GPU-resident data, for 904 B in total against 570 B, and stage 1 loses its exact top-K selection (Table 2). We therefore keep the cell scan as the default and offer the graph mode where throughput near recall 0.99 matters more than memory.

Supplementary Note 13. Throughput mode

Where the speed comes from. Table S8 adds the changes of the throughput mode one at a time (998k OpenAI vectors, $k = 10$, best of three timed runs). The first two rows keep the trellis stage 1 and show the lookup-table stage 2 alone (1.22–1.24×). The int8 stage 1 gives the largest step, because it needs fewer candidates ($K = 200$ instead of 400 at recall 0.992) and decodes nothing. The other changes remove fixed per-batch costs, which dominate at large batches: at a batch of 4,000 and $z = 1.5$, before the routing and tile changes, a query

spent 4.5 µs in the emit pass, 2.0 in stage 2, 1.8 in routing, 1.5 in stage 3, 0.95 in pairing queries with cells, 0.8 in building the lookup tables and 0.75 in the final top-K (sum of separately timed parts, without the CUDA graph).

Table S8: From the default engine to the throughput mode, one change at a time. Rows differ in batch size as marked; recall at $n_p = 384$.

Engine	Batch	K	Recall@10	QPS
Default engine	500 / 2,000	400	0.9919	22,961 / 30,600
+ uniform codebooks, lookup-table stage 2	500 / 2,000	400	0.9919	27,925 / 37,800
+ int8 stage 1 ($z = 2$)	500 / 2,000	200	0.9920	44,347 / 57,100
+ $z = 1.5$, fp16 query rotation, fused table build, int32 pairing sort, exact recomputation	4,000	200	0.9921	69,603
+ first pass $r = 0.6K$	8,000	200	0.9921	73,100
+ smaller emit tiles, 3,072-entry buffer, normalised fp16 routing table	8,000	200	0.9921	77,806
+ $\delta = 0.07$ rule	8,000	200	0.9919	82,676
+ $z = 1.25$	8,000	200	0.9917	84,396
+ $K = 175$, fp16 lookup tables	16,000	175	0.9916	87,404

What did not help. An int8 scan of every vector without cells (128 or 256 dimensions) ranks well (99.9% of the true top 10 among 400 candidates at $R = 256$), but writing the selected items cost 15 ms per 500 queries on top of a 3.5 ms product, and a threshold estimated from a 5% sample of the data cost 4–5 ms. CUDA kernels with int8 WMMA instructions, with the same inputs and outputs as the Triton kernels, gave the same candidates but took 12.6 instead of 8.5 µs per query. Keeping fp16 vectors on the GPU for an exact rerank (3,350 B per vector) thrashed the 6 GB card. Routing without sorting the probed cells broke the first pass, which needs the nearest cells first; routing by cell means projected on 256 dimensions saved 10 MB but caused more recomputations; a 2,048-entry buffer overflowed for hundreds of queries; scoring three leading tiers for all candidates and all tiers only for the best 50–80 gained nothing or lost 0.001 of recall; and predicting the threshold from query features instead of the first pass, with exact repair inside the CUDA graph, gained 0.5–5% with the trellis stage 1 and was not carried over.

Supplementary Note 14. Smaller and lower-dimensional data

229k OpenAI vectors. A simpler configuration (stage 1 only: 1-bit codes on R PCA dimensions, fused scan, CPU rerank from an fp16 host copy) reaches 0.988 at 21,348 QPS with 98 B/vector, 0.991 at 12,837 QPS with 50 B/vector and 0.9992 at 13,584 ± 492 QPS with 98 B/vector (three timed runs; an earlier single run gave 15,896). IVF-RaBitQ (9 bits) reaches 0.995 at 13,575 QPS and at most 0.998 (7,696–8,220 QPS) with at least 1,728 B/vector; CAGRA with the fp32 dataset (6,272 B/vector, multi-CTA) reaches 0.994 at 3,581 QPS and 0.999 at 952 QPS. At recall 0.999, FanarVSE is therefore 14× faster than CAGRA with 64× less GPU memory, and 1.8× faster than IVF-RaBitQ's best (0.998) with at least 18× less.

GloVe-100 (1.18M). Compressed and graph indexes both struggle on this dataset. CAGRA, swept to itopk 1024 with degree 64 and the multi-CTA algorithm, reaches at most 0.979 recall (5,623 QPS), and IVF-RaBitQ at most 0.981 (5,136 QPS; 9 bits, 4,096 lists, 2,048 probes). FAISS-GPU exact search in fp32 reaches 1.000 at 9,003 QPS with 400 B/vector and cuVS IVF-PQ 0.960 at 8,311 QPS. FanarVSE with 4-bit codes (52 B/vector) reaches 0.999 at 20,544 ± 46 QPS, 2.3× the throughput of exact search with 7.7× less memory.

CPU reference. turbovec [4] (TurboQuant on a 16-thread CPU) with 2-bit codes and a 40-candidate exact rerank reaches 1.000 at 1,924 QPS on 229k vectors and 0.9998 at 468 QPS on 998k. We list it to place FanarVSE relative to the closest CPU system, not as a same-hardware comparison; RSLM [1] is a close CPU relative of our stage-2 codec that we did not run. For a familiar reference point, FAISS HNSW32 with SQ8 storage on the same CPU (16 threads) reaches 0.950, 0.969 and 0.979 at 2,806, 1,702 and 970 QPS (ef 64/128/256) on an earlier 921k-vector version of the OpenAI DBpedia set, with 1,792 B/vector and a 430 s build.

Supplementary Note 15. Engineering variants that did not help

Table S9: Engineering variants of the GPU kernels that did not help (OpenAI-1536). Conceptual negative results are in Table 12.

Idea	Outcome
Two CUDA streams for consecutive batches	Overlapping two batches from two host threads lowered throughput by 10–35% (contention for the interpreter lock and memory).
Top-2 per block instead of the maximum	Neighbours cluster in blocks, so 80–120 blocks per query must be rescored; slower than the threshold.
Larger stage-1 tiles, larger buffer	64×16 tiles and a 64k buffer with top-K by count buckets were 4–18% slower in the engine (one micro-benchmark had suggested 10% faster).
Persistent stage-1 programs	A fixed number of programs looping over the probed (cell, block) items, compacted on the GPU, was 20–30% slower than one program per block.
One program per block for single queries	No change at batch 1 (0.57 vs. 0.56 ms for stage 1); the time is in fixed per-call costs.

References

[1] Rastislav Lenhardt et al. RSLM: Training-free vector quantization for approximate nearest neighbor search, 2026.

[2] William H. R. Equitz and Thomas M. Cover. Successive refinement of information. *IEEE Transactions on Information Theory*, 37(2):269–275, 1991.

[3] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[4] Ryan Codrai. turbovec: TurboQuant vector search. `https://github.com/ryancodrai/turbovec`, 2026. Version 1.1.2, accessed 2026-10-26.